

PCAD
Programmazione Concorrente
Algoritmi Distribuiti

Arnaud Sangnier
arnaud.sangnier@unige.it

INTERLEAVING e MUTUA ESCLUSIONE

Esecuzione concorrente

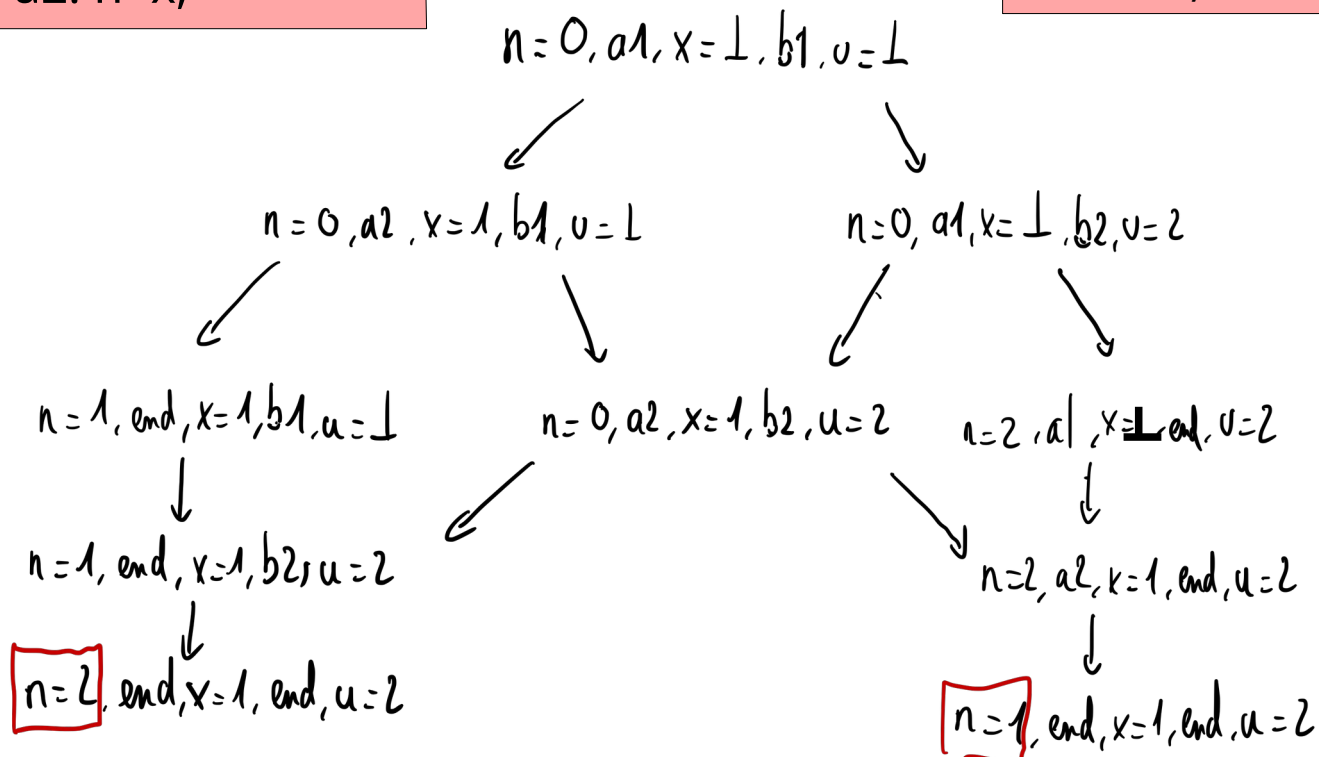
int n=0; //variabile condivisa

Process P1

```
int x;  
a1: x=1;  
a2: n=x;
```

Process P2

```
int u;  
b1: u=2;  
b2: n=u;
```



Istruzione atomiche

- Nozione **molto importante**
- La correzione dei programmi dipende degli ipotesi di atomicità
- **Un'istruzione atomica viene eseguita completamente senza essere interrotta**
- Non è sempre facile definirle
- In questo corso, supponiamo che **ogni riga dei programmi sia atomica** (tranne alcune eccezioni)
- Esempio:
 - $x=5$
 - $x=x+1$ diverso di $\begin{array}{l} \text{tmp}=x; \\ x=\text{tmp}+1; \end{array}$
 - $\text{If } (x==2) \{$
 $x=3$
 $\}$ spesso non atomica tranne test-and-est

Esecuzione concorrente

int n=0; //variabile condivisa

Process P1
a1:n=n+1

Process P2
b1: n=n+1

Se n=n+1 è atomica

Senno

int n=0; //variabile condivisa

Process P1
int tmp;
a1:tmp=n;
a2:n=tmp+1;

Process P2
int tmp2;
b1:tm2=n;
b2: n=tmp2+1

Possiamo ottenere risultati molto diversi !!!

Ipotesi di atomicità

- In realtà, gli ipotesi fatti sono poco realistici (ma permettono di ragionare, e di avere dei modelli di riferimento)
- **Difficoltà:** gli ipotesi possono cambiare a seconda del compilatore, del processore, etc

Una regola:

Evitare di manipolare più di una occorrenza di una variabile condivisa fra processi

- Invece di fare $n=n+m+5$ dove n e m sono condivisi. è meglio fare:

`tmp=n;`

`tmp=tmp+m;`

`n=tmp+5;`

`tmp:` **variabile locale al processo**

Ipotesi di atomicità

- Esistono diversi casi:
 - **registri atomici** -> operazione di lettura e di scrittura atomiche
 - **test-and-set** -> si può testare E modificare una variabile senza essere interrotti
 - **swap** -> si può scambiare il valore di un registro locale e di uno condiviso in un modo atomico
 - ...

Semantica di intreccio (interleaving)

Definizione:

Uno **stato** di un programma concorrente P è una tupla che include:

- un puntatore alle istruzioni per ciascun processo
- un valore per ciascuna delle variabile (locale e condivisa)

- Per esempio $(n=0, \text{end}, x=1, b1, u=2)$

Definizione:

Per due stati $s1$ e $s2$, si scrive **$s1 \rightarrow s2$** quando è possibile passare da $s1$ a $s2$ utilizzando una delle istruzioni puntate in $s1$

- Per esempio $(n=0, a1, x=1, b1, u=2) \rightarrow (n=2, \text{end}, x=1, \text{end}, u=2)$

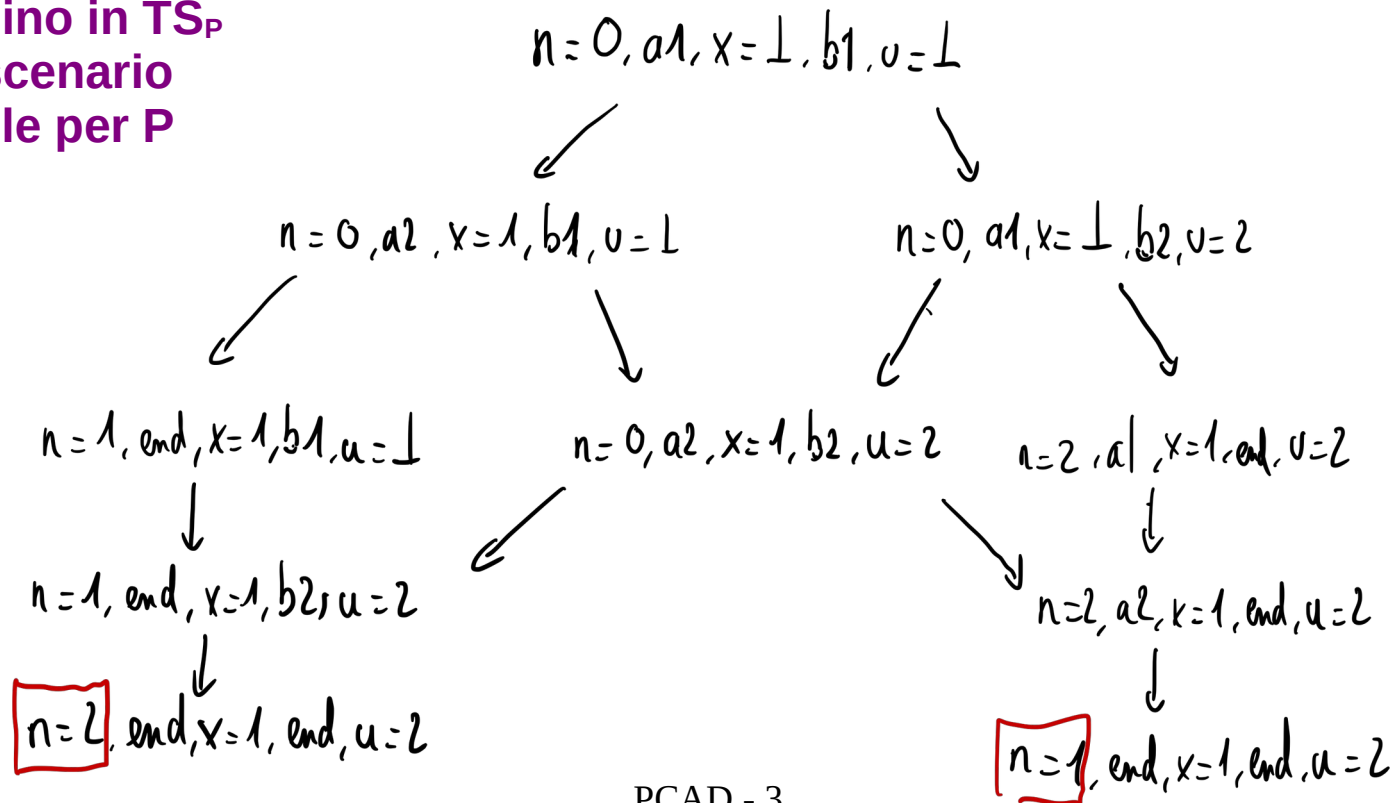
Semantica di intreccio (interleaving)

Definizione:

Il **diagramma degli stati** di P è il sistema di transizione (ovvero il grafo) $TS_P = (S, \rightarrow, s_0)$ dove:

- S è l'insieme degli stati di P
- $\rightarrow$ è la relazione di transizione inclusa in $S \times S$
- s_0 è lo stato iniziale di P

Un cammino in TS_P
è uno scenario
possibile per P



Semantica di intreccio (interleaving)

- I cammini in TS_P corrispondono a tutti gli intrecci possibili per P.
- Facciamo quindi l'ipotesi che tutti questi intrecci siano possibili
- Si tratta di un'astrazione (in pratica è molto improbabile che tutti gli intrecci siano possibili)
- A volte ci limiteremo a considerare solo gli scenari 'fair' (giusti), in cui ogni processo che può eseguire un'istruzione avrà un giorno la possibilità di farlo
- Verificheremo la correzione degli algoritmi per **TUTTI gli intrecci** (o a volte per tutti gli intrecci fair) -> questo ci permetterà di verificare più di quanto possiamo osservare in pratica, garantendo un **ragionamento robusto**

Semantica di intreccio (interleaving)

- Una semantica ragionevole:
 - Per i sistemi con un processore condiviso fra più processi
 - C'è una successione d'istruzioni di un processo, poi di un altro, ecc
 - Per i sistemi con più processori
 - Ogni processo è legato ad un processore
 - La semantica d'intreccio non corrisponde esattamente alla realtà (ci sono delle azioni parallele), ma è corretta se non ci sono conflitti sulle risorse (evitare l'accesso simultaneo ai dati condivisi)
 - Per i sistemi distribuiti
 - Diversi computer, non ci sono variabili condivise, ma dei canali di comunicazione
 - È Molto diversa dai primi due casi, ma la semantica rimane interessante se si aggiungono l'invio e la ricezione di messaggi

Un problema centrale

Il problema della mutua esclusione

permette di garantire che, in un determinato momento, un processo ha un accesso **esclusivo** a delle risorse comuni

Problema della sezione critica

Il problema:

$N \geq 2$ processi eseguono in modo continuo due sequenze d'istruzioni

- Una sezione non critica (**SNC**)
- Una sezione critica (**SC**)

in tal modo che la mutua esclusione sia garantita in SC: **ad ogni istante, c'è al massimo un processo in sezione critica**

- Per risolvere questo problema, useremo delle variabili condivise (**che non verranno usate in SC e SNC**) all'interno
 - di un pre-protocollo prima di accedere a SC
 - e di un post-protocollo dopo l'accesso a SC

Problema della sezione critica

Struttura dei programmi

Process P1

```
while(true){  
  p1: SNC  
  p2...: pre-protocollo  
  pi: SC  
  pj...: post-protocollo  
}
```

Ipotesi:

- la SC finisce sempre
- la SNC può non finire
- i due processi rimangono sempre attivi

Problema della sezione critica

Esempio

int turn=1; //variabile condivisa

```
Process P1
while(true){
p1: SNC
p2: while(turn!=1){}
p3: SC
p4: turn=2;
}
```

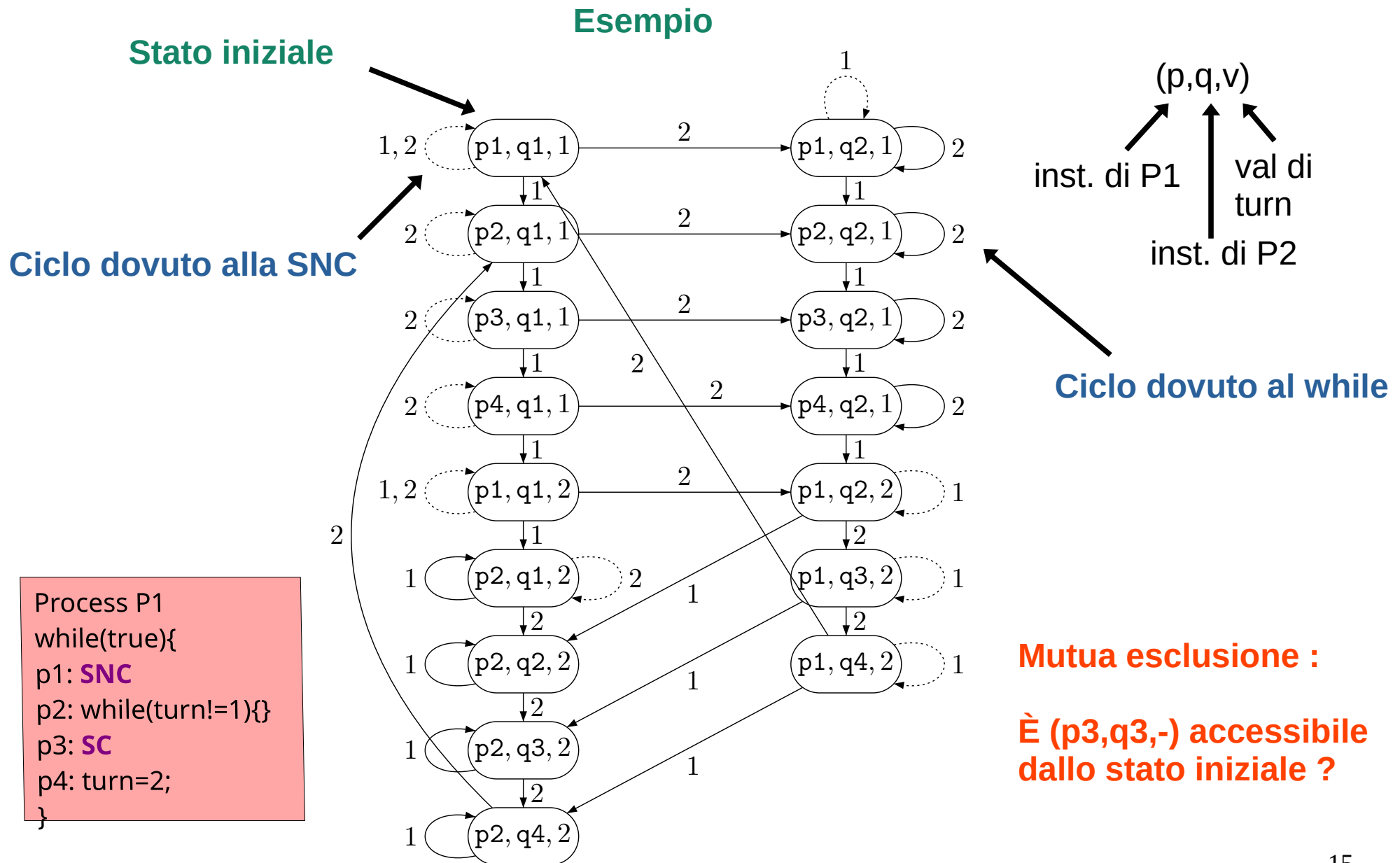
```
Process P2
while(true){
q1: SNC
q2: while(turn!=2){}
q3: SC
q4: turn=1;
}
```

Quali scenari ?

Uno **stato** di un programma concorrente P è una tupla che include:

- un puntatore d'istruzione per ciascun processo
- un valore per ciascuna delle variabile (locale e condivisa)

Problema della sezione critica



Problema della sezione critica

- Proprietà da verificare:
 - **Mutua esclusione:**
 - Al massimo, un processo in SC ad ogni momento
 - **Assenza di deadlock:**
 - Se più processi provano di accedere alla SC nello stesso momento, non devono essere bloccati nel pre-protocollo e al meno un processo deve riuscirci
 - **Assenza di 'starvation'** (carestia?):
 - Se un processo desidera accedere alla sua SC, prima o poi ci riuscirà
 - **Attesa limitata:**
 - Se un processo deve attendere per accedere alla sua SC, è possibile limitare il numero di volte in cui dovrà concedere il posto a un altro processo tramite una costante

Attesa limitata => Assenza di starvation => Assenza di deadlock

Problema della sezione critica

Esempio

`int turn=1; //variabile condivisa`

```
Process P1
while(true){
p1: SNC
p2: while(turn!=1){}
p3: SC
p4: turn=2;
}
```

```
Process P2
while(true){
q1: SNC
q2: while(turn!=2){}
q3: SC
q4: turn=1;
}
```

- Mutua esclusione ✓
- Assenza di deadlock ✓
- Assenza di starvation ✗
- Attesa limitata ✗